

FORCEBIT TICKETING.

Een support ticketing applicatie met een gelaagde architectuur.

Alex Delvoye.

GraduaatsProef Programmeren 2025–2026.



Scriptie voorgedragen tot het bekomen van de graad van
Graduaat in het Programmeren

Promotor: Emlin Oguz Inci

Co-promotor: Sacha De Maere

Instelling: Forcebit

Academiejaar: 2025–2026

Tweede examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Deze graduaatsproef is voor mij een kans geweest om de leerstof van de opleiding Graduaat Programmeren toe te passen in een project dat dicht bij een realistische werksituatie ligt. Met Forcebit Ticketing wilde ik de geziene leerstof herhalen en combineren in een oplossing voor een concreet probleem bij Forcebit: communicatie met klanten stond verspreid over telefoon en e-mail, waardoor opvolging minder overzichtelijk was. De applicatie brengt tickets, gesprekken, statussen en bijlagen samen in één centrale workflow.

Technisch bestaat het project uit een gelaagde backend in C# en ASP.NET Core, gekoppeld aan een MySQL-databank. Daarin gebruikte ik onder andere Dependency Injection, het Repository Pattern, een Service Layer, DTO-mapping en het Options Pattern. Ook de frontend is gestructureerd opgebouwd met aparte schermen, custom hooks, API-modules, formulieren, componenten, validatieschema's en utility-functies, zodat de code herbruikbaar en beter uitlegbaar blijft.

Tijdens dit project heb ik ook nieuwe onderdelen leren toepassen, zoals een mail-service, het uploaden van attachments, previews voor afbeeldingen, authenticatie met rollen en een duidelijke ticketworkflow. Daardoor voelde deze graduaatsproef voor mij als een passende afsluiter van de opleiding: een herhaling van wat ik geleerd heb, aangevuld met technieken die dichter bij echte softwareontwikkeling liggen.

Graag wil ik mijn mentor Emlin Oguz Inci bedanken voor het opvolgen van deze graduaatsproef en voor zijn feedback over de features, structuur en verdedigbaarheid van het project. Zijn opmerkingen hebben mij geholpen om niet alleen naar werkende code te kijken, maar ook naar gebruiksgemak, onderhoudbaarheid en duidelijke uitleg.

Samenvatting

Forcebit Ticketing is een webapplicatie voor het opvolgen van supporttickets. Het project bestaat uit een ASP.NET Core backend, een MySQL-databank en een Expo React Native frontend. Klanten kunnen zich registreren, inloggen, tickets aanmaken, antwoorden sturen en bijlagen uploaden. Administrators beheren tickets via een dashboard met zoeken, filtering, statusgroepen en paginatie.

De aanleiding voor dit project is het probleem dat klantcommunicatie bij Forcebit verspreid stond over telefoon en e-mail. Daardoor werd het moeilijker om supportvragen centraal op te volgen. De applicatie brengt deze communicatie samen in tickets met vaste metadata zoals titel, categorie, onderwerp en status. De volledige conversatie blijft per ticket bewaard, inclusief geuploade attachments. Afbeeldingen kunnen rechtstreeks bekeken worden, terwijl andere bestanden als download beschikbaar blijven.

De centrale onderzoeksvraag is hoe een onderhoudbare applicatie gebouwd kan worden waarin klant en administrator een duidelijke workflow volgen terwijl de functionaliteit blijft groeien. Hiervoor werd het project opgebouwd in lagen. De API-laag behandelt HTTP-verkeer en authenticatieclaims, de service-laag voert de use cases uit, de persistence-laag beheert Entity Framework Core en MySQL, en de domain-laag bevat entiteiten en regels rond tickets, berichten en attachments.

Tijdens de uitwerking lag de nadruk op Single Responsibility. Controllers verwerken geen ticketregels, services coördineren workflows en repositories vormen de grens met de databank. In de frontend werd dezelfde scheiding toegepast met aparte schermen, custom hooks, API-modules, formulieren, componenten, validatieschema's en utility-functies. Zo blijven zoeken, groeperen, pagineren, foutafhandeling en attachmentlogica herbruikbaar.

Het resultaat is een werkende eerste versie van Forcebit Ticketing met een realistische supportflow. Tickets starten als **New**, worden **Open** wanneer support antwoordt en kunnen **Closed** worden wanneer het probleem opgelost is. Wanneer een gesloten ticket opnieuw geopend wordt, gaat het terug naar **Open** en niet naar **New**. Daarnaast krijgen klanten e-mailnotificaties wanneer een administrator een antwoord plaatst of de status van een ticket wijzigt.

Belangrijke toekomstige verbeteringen zijn een uitgebreidere automatische testset, server-side paginatie en productiegerichte logging. Toch toont het project aan dat een gelaagde structuur en duidelijke verantwoordelijkheden helpen om een supportapplicatie overzichtelijk, onderhoudbaar en uitbreidbaar te maken.

Broncode: [alexdelvoye/GraduaatsProef_TicketingSystem.git](https://github.com/alexdelvoye/GraduaatsProef_TicketingSystem.git)

Summary

Forcebit Ticketing is a web application for managing support tickets. The project consists of an ASP.NET Core backend, a MySQL database, and an Expo React Native frontend. Clients can register, log in, create tickets, reply to conversations, and upload attachments. Administrators manage tickets through a dashboard with search, filtering, status groups, and pagination.

The project was created to solve a practical communication problem at Forcebit. Client communication was spread across phone calls and e-mails, which made follow-up harder to manage centrally. The application stores this communication in tickets with fixed metadata such as title, category, subject, and status. The full conversation remains linked to the ticket, including uploaded attachments. Images can be previewed directly, while other files remain available as downloads.

The central research question is how to build a maintainable ticketing application in which clients and administrators follow a clear workflow while the feature set continues to grow. The backend was therefore divided into layers. The API layer handles HTTP traffic and authentication claims, the service layer executes use cases, the persistence layer manages Entity Framework Core and MySQL, and the domain layer contains entities and rules for tickets, messages, and attachments.

Single Responsibility was an important design guideline. Controllers do not contain ticket rules, services coordinate workflows, and repositories form the boundary with the database. The frontend follows a similar structure with separate screens, custom hooks, API modules, forms, components, validation schemas, and utility functions. This keeps search, grouping, pagination, error handling, and attachment logic reusable.

The result is a working first version of a ticketing application with a realistic support workflow. Tickets start as **New**, become **Open** when support replies, and can be marked **Closed** when the issue is solved. When a closed ticket is reopened, it returns to **Open** instead of **New**. Clients also receive e-mail notifications when an administrator replies or changes the status of a ticket.

Important future improvements include a broader automated test suite, server-side pagination, and production-oriented logging. Even so, the project shows that layered structure and clear responsibilities help keep a support application organized, maintainable, and extendable.

Source code: [alexdelvoye/GraduaatsProef_TicketingSystem.git](https://github.com/alexdelvoye/GraduaatsProef_TicketingSystem.git)

Inhoudsopgave

Lijst van figuren	vii
Lijst van codefragmenten	viii
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	2
1.3 Onderzoeksdoelstelling	2
1.4 Opzet van deze graduaatsproef.	2
2 Stand van zaken	3
2.1 Ticketing als ondersteuning van supportprocessen	3
2.2 Web API en backendstructuur	4
2.3 Single Responsibility en lagen	4
2.4 Frontend state en herbruikbaarheid	4
2.5 Beveiliging en validatie	5
3 Methodologie	6
3.1 Oorspronkelijke planning.	6
3.2 Fase 1: scope en analyse.	6
3.3 Fase 2: technische basis.	7
3.4 Fase 3: frontend en feedback	7
3.5 Fase 4: controle en documentatie	7
4 Functionele analyse	8
4.1 Gebruikersrollen.	8
4.1.1 Klant	8
4.1.2 Administrator	8
4.2 Ticketstatussen	9
4.3 Bijlagen	9
4.4 Notificaties	9
5 Backendarchitectuur en beveiliging	10
5.1 Gelaagde structuur.	10
5.2 Controllers en DTOs	11
5.3 Services als use cases	11
5.4 Domain rules.	12
5.5 Authenticatie en autorisatie	12

6	Databank en persistentie	13
6.1	Datamodel	13
6.2	DbContext en configuraties	14
6.3	Enumwaarden en relaties	14
6.4	Repository pattern	14
7	Frontendarchitectuur en gebruikersinterface	15
7.1	Schermen, hooks en componenten	15
7.2	Dashboardlogica	16
7.3	Paginatie en hergebruik	16
7.4	Ticketgroepen	17
7.5	Conversatieweergave	17
8	Aanvullende functionaliteiten	18
8.1	E-mailnotificaties met Brevo	18
8.2	Bijlagen en uploadlimiet	19
8.3	Documentatie	19
9	Resultaten en evaluatie	20
9.1	Werkend resultaat	20
9.2	Onderhoudbaarheid	20
9.3	Beperkingen en verdere ontwikkeling	21
10	Conclusie	22
	Bibliografie	23

Lijst van figuren

2.1	Samenvatting stand van zaken	5
4.1	Klantenlijst in het adminoverzicht	9
5.1	Backendlagen	11
6.1	Databankschema	13
7.1	Frontendarchitectuur	15
7.2	Ticketfilters	16
7.3	Ticketconversatie	17
8.1	Bijlagen selecteren	19

Lijst van codefragmenten

5.1	Statuscontrole in TicketRules	12
6.1	DbContext met configuraties	14
7.1	Generieke paginatiehook	16
8.1	E-mail bij adminantwoord	18

1

Inleiding

Supportcommunicatie is in veel organisaties een terugkerende uitdaging. Klanten melden problemen, stellen vragen of sturen bestanden door, maar die informatie staat vaak verspreid over telefoon, e-mail en losse documenten. Daardoor is het niet altijd duidelijk welke vragen nieuw zijn, welke al behandeld worden en welke opgelost zijn. Forcebit Ticketing vertrekt vanuit die nood en centraliseert supportvragen in tickets met een titel, categorie, onderwerp, status en volledige conversatie.

Op die manier kan de klant de stand van zaken opvolgen en kan de administrator tickets beheren vanuit een overzichtelijk dashboard. Het project is daarom niet alleen een oefening in functionaliteit, maar ook in structuur: de applicatie moet bruikbaar blijven wanneer er extra schermen, regels of workflows bijkomen.

1.1. Probleemstelling

Wanneer supportvragen niet centraal worden bijgehouden, verliest zowel klant als administrator overzicht. De klant weet niet altijd of een probleem al wordt opgevolgd. De administrator moet zoeken in verschillende berichten, bestanden of oude mails. Ook bijlagen zoals screenshots zijn minder bruikbaar wanneer ze los van de oorspronkelijke vraag worden doorgestuurd.

Een ticketingsysteem brengt structuur in die communicatie, maar het bouwen ervan is meer dan enkel schermen maken. De applicatie moet duidelijke regels bevatten rond rechten, antwoorden, statusovergangen en bestandsopslag. Zonder duidelijke structuur komt die logica snel verspreid terecht in controllers, schermen of losse hulpfuncties, waardoor latere aanpassingen moeilijker worden.

1.2. Onderzoeksvraag

De centrale onderzoeksvraag van deze graduaatsproef luidt:

Hoe kan een onderhoudbare ticketing applicatie gebouwd worden waarin klant en administrator een duidelijke supportworkflow volgen, met aandacht voor Single Responsibility, gelaagde structuur en herbruikbare frontendlogica?

Daaruit volgen enkele deelvragen:

- Hoe krijgen HTTP, workflow, databanktoegang en business rules elk een duidelijke plaats in de backend?
- Hoe blijft de frontend leesbaar wanneer zoeken, filtering, paginatie en uploads toegevoegd worden?
- Hoe kan de ticketworkflow met New, Open en Closed logisch en consequent afgedwongen worden?
- Hoe kan documentatie technische keuzes, verantwoordelijkheden en uitbreidingspunten vastleggen?

1.3. Onderzoeksdoelstelling

Het doel is een werkend prototype te bouwen van een support ticketing applicatie. Klanten moeten tickets kunnen aanmaken, beantwoorden, sluiten en heropenen. Administrators moeten tickets kunnen zoeken, filteren, beantwoorden en van status veranderen. Succes wordt niet alleen bepaald door functionaliteit: de codebase moet ook onderhoudbaar blijven. Daarom worden lagen, Single Responsibility, DTO's, domeinregels, repositories, hooks en herbruikbare componenten bewust ingezet.

1.4. Opzet van deze graduaatsproef

De rest van deze graduaatsproef is opgebouwd rond analyse, uitwerking en evaluatie:

- Hoofdstuk 2: concepten rond ticketing, web API's, gelaagde architectuur, frontend state en basisbeveiliging.
- Hoofdstuk 3: de gevolgde aanpak tijdens de ontwikkeling.
- Hoofdstuk 4: gebruikersrollen, ticketstatussen, bijlagen en notificaties.
- Hoofdstuk 5: de backendarchitectuur, lagen, patronen en autorisatie.
- Hoofdstuk 6: de MySQL databank, EF Core configuratie en relaties.
- Hoofdstuk 7: de frontendstructuur, herbruikbare hooks, componenten en schermen.
- Hoofdstuk 8: e-mailnotificaties, bijlagen en documentatie.
- Hoofdstuk 9: het resultaat, de evaluatie en de belangrijkste beperkingen.
- Hoofdstuk 10: het antwoord op de onderzoeksvraag.

2

Stand van zaken

Dit hoofdstuk bespreekt de belangrijkste concepten achter Forcebit Ticketing. De focus ligt niet op een volledige theoretische bespreking van elk framework. De nadruk ligt op de onderdelen die nodig zijn om de gemaakte keuzes te plaatsen: ticketing, web API's, gelaagde architectuur, frontend state en beveiliging. Die concepten keren later terug in de functionele analyse en in de technische implementatie.

2.1. Ticketing als ondersteuning van supportprocessen

Een ticketingsysteem heeft als doel om vragen, problemen, antwoorden en bijlagen centraal bij te houden. In plaats van losse communicatie bevat elk ticket de context van een vraag. Binnen IT service management sluiten zulke tickets aan bij het opvolgen van incidenten en service requests, waarbij structuur en traceerbaarheid belangrijk zijn (AXELOS, [2019](#); ISO/IEC, [2018](#)). Voor dit project betekent dat: een klant maakt een ticket aan, support reageert, de status evolueert en alle berichten blijven gekoppeld aan hetzelfde ticket.

Een belangrijk voordeel is traceerbaarheid. Een ticketingtool brengt aanvragen samen op een centrale plaats, zodat opvolging en communicatie niet over losse kanalen verspreid blijven (Atlassian, [2026](#)). Wanneer een ticket later opnieuw bekeken wordt, staat de geschiedenis op een plaats. Dat is vooral nuttig wanneer een probleem meerdere antwoorden of bijlagen bevat. In Forcebit Ticketing wordt daarom de eerste beschrijving van de klant niet als apart tekstveld op het ticket bewaard, maar als eerste bericht in de conversatie. Zo volgt elk ticket hetzelfde conversatiemodel vanaf het begin.

2.2. Web API en backendstructuur

ASP.NET Core is een framework om webapplicaties en API's te bouwen. De documentatie beschrijft onder meer controllers, dependency injection, middleware en configuratie als vaste onderdelen van de applicatiestructuur (Microsoft, 2026a). Die onderdelen passen bij dit project omdat de backend verschillende verantwoordelijkheden heeft: HTTP requests verwerken, authenticatie controleren, services aanroepen en foutafhandeling centraliseren.

Entity Framework Core wordt gebruikt als object-relational mapper. Het laat toe om met C# entiteiten te werken en die via configuraties en migraties aan een relationele databank te koppelen (Microsoft, 2026c). Voor Forcebit Ticketing is dat nuttig omdat tickets, berichten, bijlagen en gebruikers duidelijke relaties hebben. De databankstructuur kan daardoor aansluiten bij de domeinobjecten in de code.

2.3. Single Responsibility en lagen

Single Responsibility betekent dat een klasse of module een duidelijke hoofdreden heeft om te veranderen. Robert C. Martin, auteur van *Clean Architecture*, koppelt dit principe aan onderhoudbare softwarestructuur: code blijft beter aanpasbaar wanneer verantwoordelijkheden niet door elkaar lopen (Martin, 2017). In een ticketingapplicatie is dit belangrijk omdat dezelfde workflow meerdere technische onderdelen raakt. Een antwoord toevoegen betekent bijvoorbeeld: rechten controleren, statusregels toepassen, het bericht bewaren, bijlagen koppelen en eventueel een e-mail sturen.

Als al die logica rechtstreeks in een controller zou staan, wordt de code moeilijk te onderhouden. Daarom verdeelt een gelaagde structuur de verantwoordelijkheden: controllers behandelen HTTP en geven de nodige gegevens door via Data Transfer Objects (DTOs), objecten die volgens Fowler data over een grens transporteren (Fowler, 2003). Services voeren use cases uit: afgebakende gebruikersdoelen of systeemgedrag dat door de applicatie wordt ondersteund (Jacobson e.a., 1992). Repositories verzorgen databanktoegang. Domain rules bevatten businessregels zoals toegang en statusovergangen. De lagen maken de applicatie niet automatisch correct, maar ze geven wel een duidelijke plaats aan elke soort logica.

2.4. Frontend state en herbruikbaarheid

De frontend is gebouwd met Expo React Native. React Native is bedoeld om interfaces te bouwen met componenten, terwijl Expo extra tooling aanbiedt om sneller te ontwikkelen en ook web te ondersteunen (Expo, 2026; Meta Open Source, 2026). React Navigation wordt gebruikt voor schermnavigatie en web routes (React Navigation, 2026).

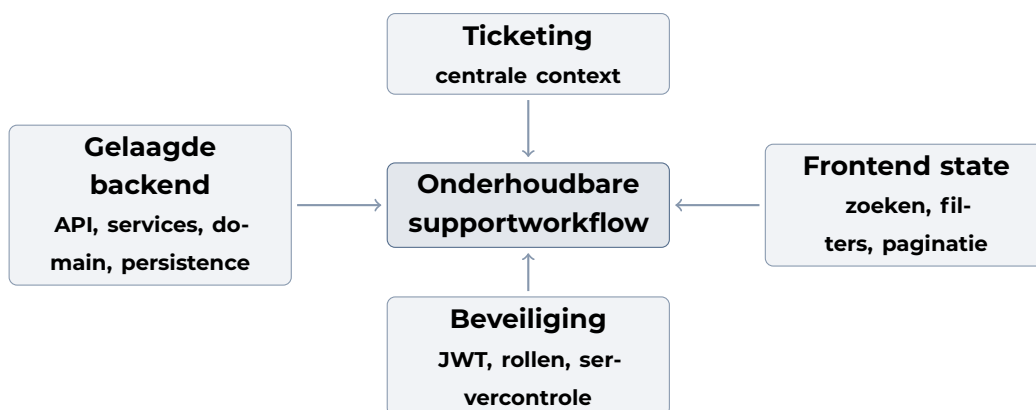
In moderne frontendapplicaties is state management een belangrijk aandachtspunt. In Forcebit Ticketing worden ruwe API-resultaten bewaard als bron van waar-

heid. Zoekresultaten, filters, tellingen, groepen en paginatie worden daarvan afgeleid. Dit wordt *derived state* genoemd. Het voorkomt dat meerdere lijsten apart bijgehouden moeten worden en uit sync kunnen raken. Herbruikbare hooks, componenten en utility-functies zorgen er daarnaast voor dat dezelfde logica niet opnieuw in elk scherm geschreven moet worden.

2.5. Beveiliging en validatie

Omdat tickets klantgegevens en bijlagen kunnen bevatten, mag de frontend niet de enige beveiligingslaag zijn. De backend moet opnieuw controleren of de gebruiker een ticket mag zien, beantwoorden of wijzigen. OWASP benadrukt dat toegangscontrole aan serverzijde moet worden afgedwongen en niet mag afhangen van client-side controles (OWASP Foundation, 2026).

Forcebit Ticketing gebruikt JWT authenticatie en role-based authorization. Een JSON Web Token is een compact tokenformaat waarmee claims tussen partijen kunnen worden doorgegeven (Jones e.a., 2015). In dit project bevat het token onder andere de gebruikersidentiteit en rol. Frontendvalidatie met Yup helpt de gebruiker sneller fouten te zien, maar backendvalidatie blijft noodzakelijk voor betrouwbaarheid en veiligheid.



Figuur 2.1: Visuele samenvatting van de stand van zaken. De literatuurstudie toont hoe ticketing, een gelaagde backend, herbruikbare frontendlogica en beveiliging samen bijdragen aan een onderhoudbare supportworkflow.

3

Methodologie

Dit hoofdstuk beschrijft hoe Forcebit Ticketing werd aangepakt. Het oorspronkelijke plan van aanpak was een compacte planning voor een eerste prototype. Tijdens de uitvoering werd die planning aangepast wanneer mentorfeedback, technische keuzes of onderhoudbaarheid daar aanleiding toe gaven. Per fase worden doel, werkwijze en resultaat samengevat.

3.1. Oorspronkelijke planning

Het oorspronkelijke plan vertrok van een korte ontwikkelperiode waarin eerst de basisstructuur en daarna de belangrijkste functionaliteit werd gebouwd:

Dag 1: requirements, databankontwerp en wireframes

Dag 2: backendproject, MySQL databank en modellen

Dag 3: authenticatie en rollen

Dag 4: ticket CRUD

Dag 5: berichten en antwoorden

Dag 6: React Native login, registratie en navigatie

Dag 7: klantenschermen voor overzicht, aanmaken en detail

Dag 8: adminschermpjes

Dag 9: bestandsuploads en e-mailnotificaties

Dag 10: testen, fouten oplossen, documentatie en demo

De uitvoering volgde deze planning op hoofdlijnen, maar sommige onderdelen kregen meer aandacht dan eerst voorzien. Vooral chatweergave, zoekfilters, paginatie, responsive layout en documentatie werden later uitgebreid.

3.2. Fase 1: scope en analyse

Doel. De eerste fase moest duidelijk maken welke rollen, gegevens en workflows nodig waren.

Werkwijze. Hiervoor werden requirements en wireframes opgesteld vanuit drie perspectieven: klant, administrator en systeem.

Resultaat. De kern werd vastgelegd als gebruikers, tickets, berichten en bijlagen. De statusflow werd later verfijnd van Open, In Progress en Closed naar New, Open en Closed, waarbij New alleen voor pas aangemaakte tickets dient.

3.3. Fase 2: technische basis

Doel. De tweede fase moest een backend opleveren waarop de ticketflow betrouwbaar kon steunen.

Werkwijze. Er werd een prototype gebouwd met ASP.NET Core, Entity Framework Core en MySQL. De code werd meteen gelaagd opgezet om verantwoordelijkheden te scheiden.

Resultaat. De belangrijkste deliverables waren entities, DTOs, controllers, repositories, services, configuratie, migraties, JWT-authenticatie, centrale foutafhandeling en het Options pattern (Microsoft, [2026d](#)).

3.4. Fase 3: frontend en feedback

Doel. De derde fase moest de backend bruikbaar maken voor klanten en administrators.

Werkwijze. De wireframes werden omgezet naar frontend-schermen. Expo werd gebruikt als ontwikkelplatform (Expo, [2026](#)). Formik werd gebruikt voor formulierstate en Yup voor schema-validatie (Formik, [2026](#); Quense, [2026](#)). Herbruikbare logica werd verplaatst naar hooks, API modules, componenten, stijlen en utilities.

Resultaat. De basisflow werd aangevuld met mentorfeedback: chatbubbels, afbeeldingspreview, uploadlimiet, responsive layout, browsernavigatie, zoekvelden, filters en paginatie.

3.5. Fase 4: controle en documentatie

Doel. De laatste fase moest nagaan of de applicatie ook bij realistische data en langere gesprekken bruikbaar en onderhoudbaar bleef.

Werkwijze. Er werd realistische testdata toegevoegd voor meerdere klanten, tickets en langere gesprekken. Daarna werden zoekfilters, statusgroepen, paginatie, database-startup en documentatie gecontroleerd.

Resultaat. Code, comments en README-bestanden werden opgeschoond. Zo sluit deze fase aan bij de onderzoeksvraag rond onderhoudbaarheid: de applicatie blijft niet alleen werkend, maar ook uitbreidbaar en onderhoudbaar.

4

Functionele analyse

Dit hoofdstuk beschrijft wat de applicatie functioneel moet doen. De focus ligt op de gebruikersrollen, de belangrijkste schermen en de ticket workflow.

4.1. Gebruikersrollen

De applicatie heeft twee rollen: klant en administrator. Een klant gebruikt de applicatie om supportvragen te melden en op te volgen. Een administrator gebruikt de applicatie om tickets te beheren en te beantwoorden.

4.1.1. Klant

Een klant kan zich registreren, inloggen en een profiel beheren. Daarna kan de klant tickets aanmaken. Bij een ticket kiest de klant een categorie, onderwerp en titel. De beschrijving die de klant invult, wordt opgeslagen als eerste bericht in de conversatie. De klant kan later antwoorden en bijlagen toevoegen zolang het ticket niet gesloten is.

Een klant kan een eigen ticket sluiten wanneer het probleem opgelost is. Als het probleem toch terugkomt, kan de klant het ticket opnieuw openen. Het ticket krijgt dan de status `Open`, niet `New`, omdat het geen nieuw ticket meer is.

4.1.2. Administrator

Een administrator kan alle tickets bekijken. Het adminoverzicht bevat een klantenlijst en een ticketlijst. Omdat die lijsten snel lang worden, zijn er zoekvelden, filters en paginatie toegevoegd. Administrators kunnen tickets beantwoorden en de status wijzigen naar `Open` of `Closed`.

4.2. Ticketstatussen

De ticketstatussen zijn bewust beperkt tot drie waarden:

- **New:** het ticket is aangemaakt, maar support heeft nog niet geantwoord.
- **Open:** de conversatie is actief of een gesloten ticket werd heropend.
- **Closed:** het ticket is afgehandeld en er kan niet meer geantwoord worden.

Deze namen zijn gekozen omdat ze duidelijker zijn voor de gebruiker. **New** toont dat een ticket nog wacht op de eerste supportreactie. **Open** toont dat er een actieve conversatie loopt. **Closed** toont dat het probleem als opgelost beschouwd wordt.

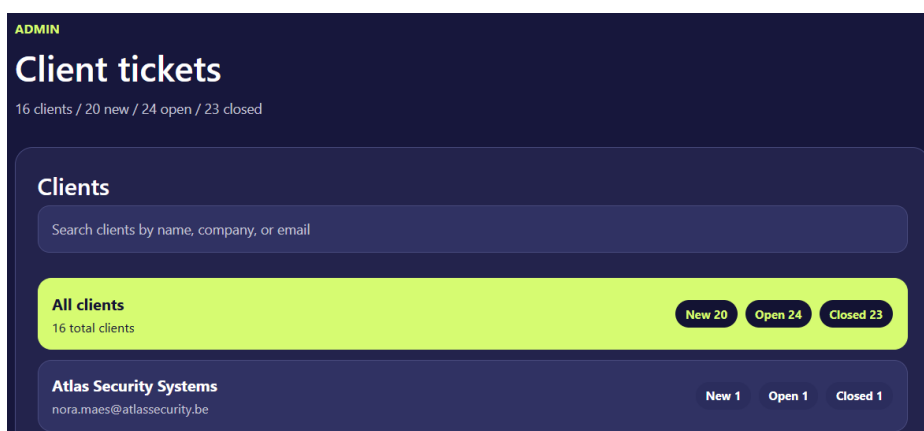
4.3. Bijlagen

Bijlagen kunnen toegevoegd worden bij ticketcreatie en bij antwoorden. De toegelaten extensies zijn PNG, JPG, JPEG, PDF en ZIP. De uploadlimiet is 20 MB per bericht. De frontend toont hoeveel van die limiet al gebruikt is. De backend controleert dezelfde limiet opnieuw.

Afbeeldingen krijgen een preview in de conversatie. Screenshots komen vaak voor bij supportvragen. De gebruiker hoeft een afbeelding dan niet eerst te downloaden om te zien wat erin staat.

4.4. Notificaties

Wanneer een administrator antwoordt of de status wijzigt, krijgt de klant een e-mailnotificatie. Klantacties sturen geen e-mail naar administrators. Administrators gebruiken het dashboard als werkqueue. Dit voorkomt dat een mailbox overspoeld wordt wanneer veel klanten antwoorden.



Figuur 4.1: Klantenlijst in het adminoverzicht met een zoekveld en paginatie.

5

Backendarchitectuur en beveiliging

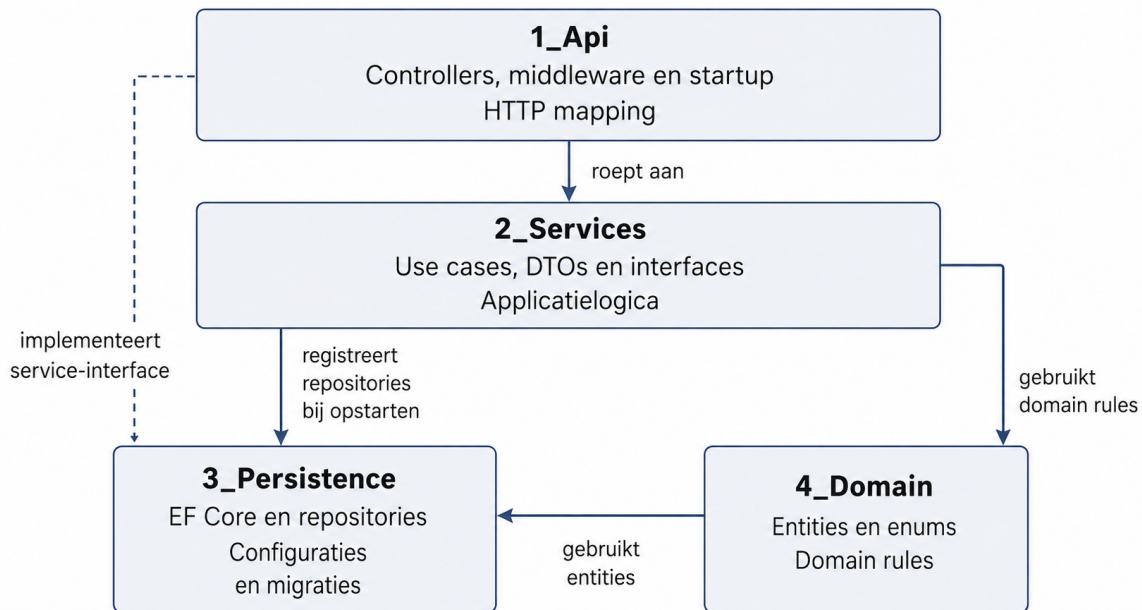
Dit hoofdstuk bespreekt de backend van Forcebit Ticketing. De nadruk ligt op de gelaagde structuur, de verdeling van verantwoordelijkheden, de gebruikte patronen en de manier waarop toegang tot tickets wordt gecontroleerd.

5.1. Gelaagde structuur

De backend bestaat uit vier projecten. Elke laag heeft een duidelijke hoofdtaak:

- **1_Api:** ontvangt HTTP requests, leest JWT claims, mapt request data en registreert dependencies.
- **2_Services:** voert de use cases uit, gebruikt DTOs en bevat interfaces voor externe onderdelen zoals repositories en e-mail.
- **3_Persistence:** bevat Entity Framework Core, repositories, configuraties en migraties.
- **4_Domain:** bevat entities, enums en domain rules. Deze laag hangt niet af van de andere projectlagen.

De requestflow loopt meestal van API naar Services en daarna naar repositories. De projectreferenties tonen ook hoe de lagen technisch gekoppeld zijn: de API kent Persistence voor dependency injection, terwijl Persistence de repository interfaces uit Services implementeert.



Figuur 5.1: Backendlagen en belangrijkste projectafhankelijkheden.

5.2. Controllers en DTOs

Controllers blijven bewust dun. Zij lezen routeparameters, request bodies, form-data en JWT claims, maar voeren zelf geen ticketlogica uit. Daarna geven ze DTOs door aan de service-laag. Dit zorgt ervoor dat HTTP-details niet verspreid raken door de rest van de applicatie.

In dit project wordt het DTO-principe vooral gebruikt tussen de HTTP-laag en de service-laag: controllers vertalen requests naar DTOs, waarna services verder werken met een eenvoudiger contract.

Een voorbeeld is uploadfunctionaliteit. ASP.NET Core ontvangt bestanden via HTTP. In de controller is dat `IFormFile`, maar de service-laag gebruikt `FileUploadRequest`. Daardoor blijft de service niet afhankelijk van een specifiek HTTP-type.

5.3. Services als use cases

`TicketService` is de belangrijkste use-case klasse. Deze service maakt tickets aan, laadt ticketdetails, voegt berichten toe, wijzigt statussen, bewaart bijlagen en beslist wanneer een e-mail verstuurd wordt. De klasse coordineert dus meerdere onderdelen, maar laat HTTP-antwoorden over aan controllers en databasequeries aan repositories.

Dit is een voorbeeld van Single Responsibility op projectniveau. De service heeft als hoofdtak het uitvoeren van ticket use cases. De concrete opslag van bestanden, het verzenden van e-mails en het ophalen van databankgegevens zitten in aparte services of repositories.

5.4. Domain rules

Businessregels rond tickets staan in de domain-laag. Daardoor kunnen controllers en services dezelfde regels gebruiken zonder ze te kopiëren. De statusregel hieronder toont bijvoorbeeld dat New alleen door ticketcreatie mag ontstaan. Een gebruiker kan een bestaand ticket dus niet terug naar New zetten.

```
1 public static bool CanChangeStatus(  
2     Ticket ticket,  
3     Guid userId,  
4     UserRole userRole,  
5     TicketStatus newStatus)  
6 {  
7     if (newStatus == TicketStatus.New)  
8     {  
9         return false;  
10    }  
11  
12    if (userRole == UserRole.Admin)  
13    {  
14        return true;  
15    }  
16  
17    if (userRole != UserRole.Client || ticket.ClientId != userId)  
18    {  
19        return false;  
20    }  
21  
22    return newStatus == TicketStatus.Closed  
23        ? ticket.Status != TicketStatus.Closed  
24        : newStatus == TicketStatus.Open && ticket.Status == TicketStatus.Closed;  
25 }
```

Codefragment 5.1: Statuscontrole in TicketRules.

5.5. Authenticatie en autorisatie

De applicatie gebruikt JWT-authenticatie. Na login bevat het token onder andere de gebruikersidentiteit en rol. Controllers kunnen routes afschermen met rollen, maar de service-laag controleert ook tickettoegang via domain rules. Dat is belangrijk omdat een klant alleen eigen tickets mag zien, terwijl een administrator alle tickets mag beheren.

De combinatie van JWT, rollen en domain rules voorkomt dat de frontend de enige beveiligingslaag wordt. Zelfs als een gebruiker manueel een API-call zou proberen, blijft de backend controleren of die gebruiker de gevraagde actie mag uitvoeren.

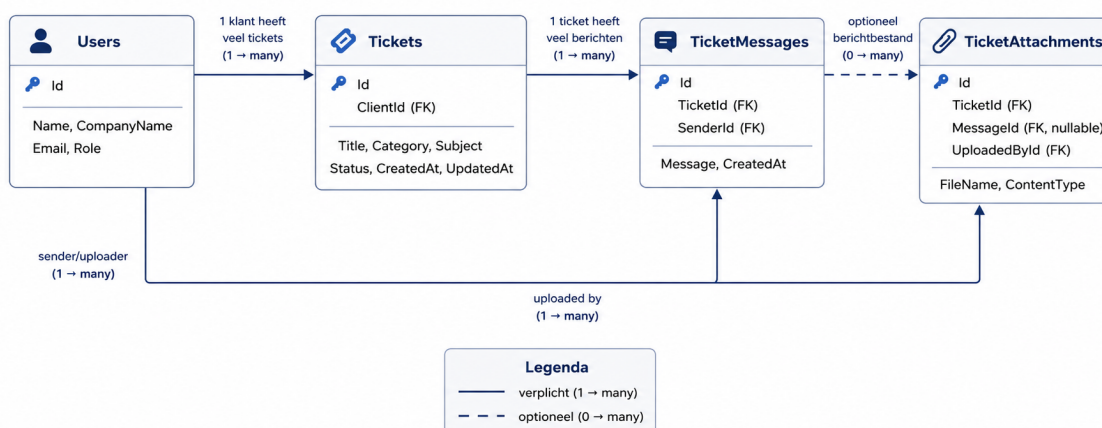
6

Databank en persistentie

Dit hoofdstuk bespreekt hoe data wordt bewaard. Forcebit Ticketing gebruikt een MySQL-databank met Entity Framework Core als Object-Relational Mapper (ORM). EF Core werkt met C# entiteiten en vertaalt queries naar databankbewerkingen, zodat de applicatie met objecten kan werken in plaats van rechtstreeks met SQL-tabellen (Microsoft, 2026c).

6.1. Datamodel

Het datamodel draait rond vier hoofdtabellen: gebruikers, tickets, berichten en bijlagen. Een gebruiker kan meerdere tickets hebben. Een ticket heeft meerdere berichten. Bijlagen horen bij een ticket en kunnen ook aan een specifiek bericht gekoppeld worden.



Figuur 6.1: Databankschema van de belangrijkste tabellen en relaties.

6.2. DbContext en configuraties

AppDbContext is de centrale EF Core klasse. De DbSet properties tonen welke tabellen de applicatie gebruikt. De detailconfiguratie staat in aparte configuratieklassen, zodat de context kort blijft en elke tabelmapping apart onderhoudbaar is.

```
1 public DbSet<User> Users => Set<User>();  
2 public DbSet<Ticket> Tickets => Set<Ticket>();  
3 public DbSet<TicketMessage> TicketMessages => Set<TicketMessage>();  
4 public DbSet<TicketAttachment> TicketAttachments => Set<TicketAttachment>();  
5  
6 protected override void OnModelCreating(ModelBuilder modelBuilder)  
7 {  
8     base.OnModelCreating(modelBuilder);  
9     modelBuilder.ApplyConfigurationsFromAssembly(  
10         typeof(AppDbContext).Assembly);  
11 }
```

Codefragment 6.1: AppDbContext laadt configuraties uit de persistence-laag.

6.3. Enumwaarden en relaties

Ticketstatus, categorie en onderwerp zijn enums in de domain-laag. Een enum groepeerde vaste, benoemde waarden binnen een eigen type (Microsoft, 2026b). In de databank worden deze waarden als tekst opgeslagen. Dat maakt de inhoud leesbaarder tijdens testen en demo's. Een rij met status Open is duidelijker dan een numerieke waarde zoals 1.

De relatie tussen ticket en klant gebruikt DeleteBehavior.Restrict. Daardoor kan een gebruiker niet zomaar verwijderd worden als tickets nog naar die gebruiker verwijzen. Voor een supportapplicatie is dat logisch, omdat ticketgeschiedenis belangrijk blijft.

6.4. Repository pattern

Repositories schermen querydetails af van de service-laag. TicketService vraagt bijvoorbeeld een ticketdetail op, maar hoeft niet te weten welke relaties mee geladen worden. Dat maakt de service beter leesbaar: de service beschrijft de workflow, de repository beschrijft de databankquery.

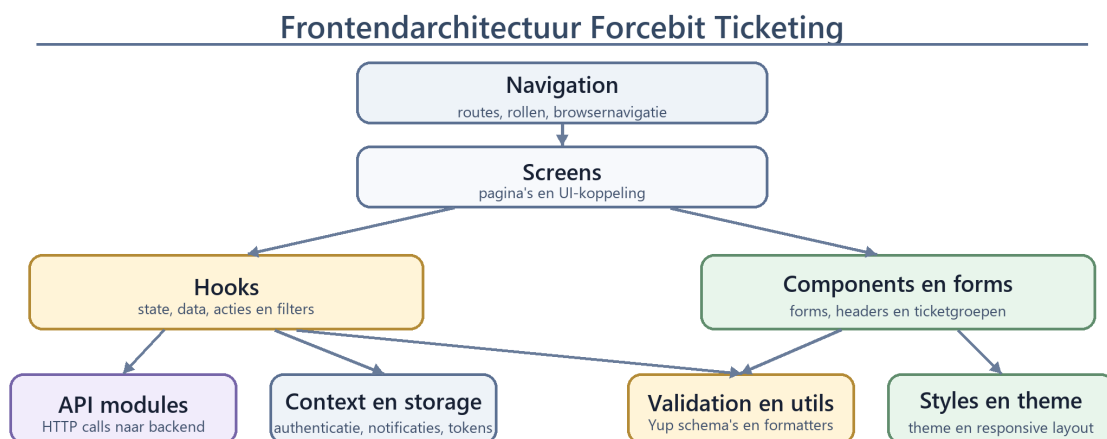
7

Frontendarchitectuur en gebruikersinterface

Dit hoofdstuk bespreekt de frontend. De applicatie is gebouwd met Expo React Native. De frontend is opgesplitst in screens, hooks, components, API modules, utilities, validation en styles. Die structuur maakt het makkelijker om gedrag te hergebruiken en schermen overzichtelijk te houden.

7.1. Schermen, hooks en componenten

Screens bepalen welke onderdelen op het scherm verschijnen. Hooks beheren gedrag zoals data laden, zoeken, filteren en pagineren. Components zijn herbruikbare UI-blokken. API modules bevatten de communicatie met de backend.



Figuur 7.1: Schema van de frontendstructuur met navigation, screens, hooks, herbruikbare componenten en ondersteunende modules.

7.2. Dashboardlogica

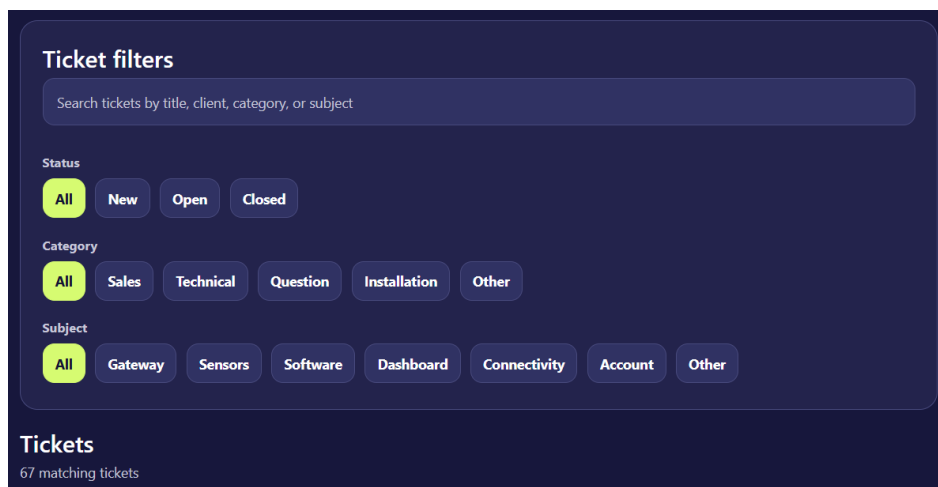
De dashboards bewaren de ruwe tickets als bron van waarheid. Daarna worden zoekresultaten, filters, statusgroepen, tellingen en pagina's afgeleid. Dit vermindert de kans op inconsistente lijsten. Als de ruwe ticketlijst verandert, worden de afgeleide resultaten opnieuw berekend.

Voor de adminpagina ziet de pipeline er conceptueel zo uit:

klanten en tickets → zoeken → filters → statusgroepen → paginatie

Voor de klantpagina is de pipeline eenvoudiger:

tickets → zoeken en filteren → statusgroepen → paginatie



Figuur 7.2: Ticketoverzicht met zoekveld en filters voor status, categorie en onderwerp.

7.3. Paginatie en hergebruik

Paginatie is gecentraliseerd in `usePagination`. De hook is generiek gemaakt, zodat hij niet alleen voor tickets werkt, maar ook voor gesprekken en klantenlijsten. Daardoor hoeft dezelfde paginalogica niet op meerdere plaatsen opnieuw geschreven te worden.

```

1 export function usePagination<T>(  

2   items: T[],  

3   { pageSize, initialPage = "first", resetKey }: UsePaginationOptions,  

4 ) {  

5   const totalPages = Math.max(1, Math.ceil(items.length / pageSize));  

6   const [currentPage, setCurrentPage] = useState(1);  

7  

8   const pageItems = useMemo(  

9     () => items.slice(startIndex, endIndex),  

10    [endIndex, items, startIndex],  

11  );  

12  

13   return { pageItems, currentPage, totalPages };  

14 }

```

Codefragment 7.1: Vereenvoudigd voorbeeld van de generieke paginatiehook.

7.4. Ticketgroepen

De klant- en adminpagina gebruiken statusgroepen voor New, Open en Closed. TicketGroupSection toont een groep tickets en gebruikt intern dezelfde paginatiehook. Elke groep heeft eigen paginatie. Bladeren in gesloten tickets verandert dus niet de pagina van nieuwe tickets.

7.5. Conversatieweergave

In het ticketdetail wordt de conversatie weergegeven met berichten aan verschillende kanten. Adminberichten en klantberichten krijgen ook een andere achtergrondkleur. Dit lijkt meer op chatapplicaties zoals WhatsApp of Messenger en maakt een lange conversatie sneller scanbaar.



Figuur 7.3: Ticketdetail met chatbubbels, visueel onderscheid tussen klant en admin en afbeeldingspreview.

8

Aanvullende functionaliteiten

Naast de basisflow bevat de applicatie enkele extra functies die belangrijk zijn in een realistische supportomgeving. Dit hoofdstuk bespreekt e-mailnotificaties, bijlagen en documentatie.

8.1. E-mailnotificaties met Brevo

Wanneer een administrator antwoordt of een status wijzigt, krijgt de klant een e-mail. De backend gebruikt hiervoor Brevo, een provider die via zijn API transactionele berichten kan versturen (Brevo, 2026). `TicketService` beslist wanneer een notificatie nodig is. `EmailService` bouwt de e-mail en stuurt ze naar de provider. Daardoor blijft de ticketlogica gescheiden van provider-specifieke code.

```
1 if (role == UserRole.Admin && ticket.Client != null)
2 {
3     await _emailService.SendTicketMessageEmailAsync(
4         ticket.Client.Email,
5         ticket.Title,
6         request.Message,
7         role.ToString(),
8         emailAttachments);
9 }
```

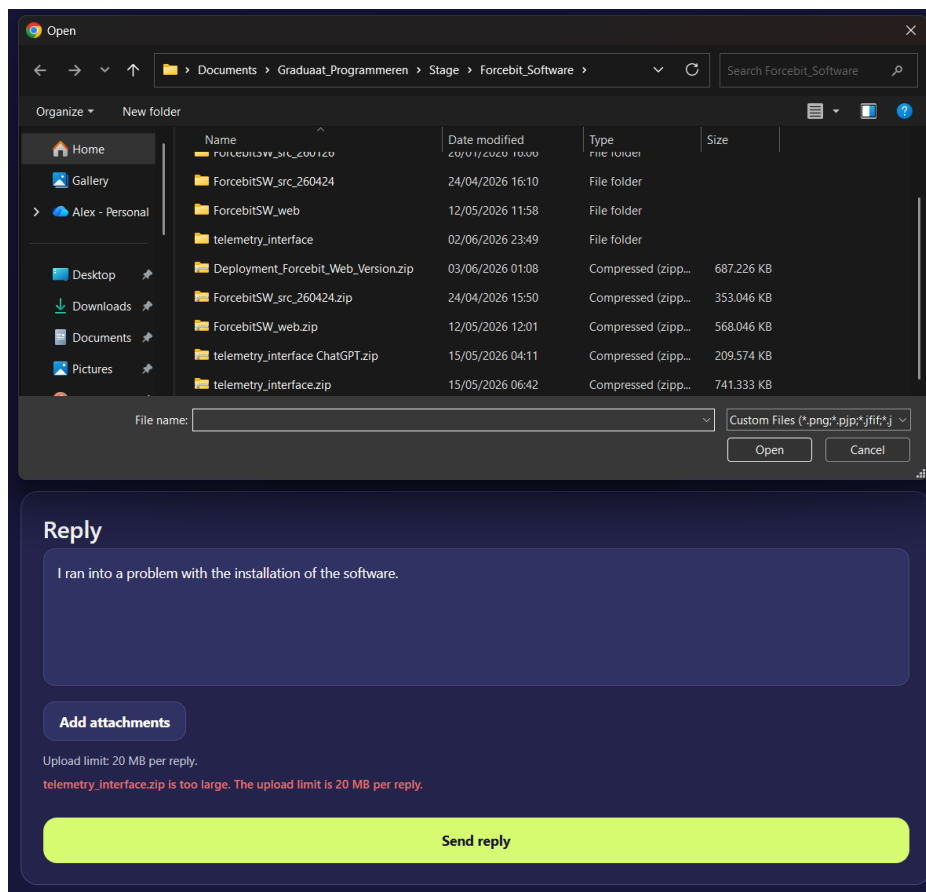
Codefragment 8.1: Een adminantwoord triggert een e-mail naar de klant.

In lokale ontwikkeling mag de applicatie ook werken zonder Brevo API key. Bij een ontbrekende sleutel logt `EmailService` een waarschuwing en slaat het verzenden over. De ticketflow loopt daardoor niet vast door e-mailconfiguratie.

8.2. Bijlagen en uploadlimiet

Bijlagen kunnen toegevoegd worden bij ticketcreatie en bij antwoorden. De frontend gebruikt Expo DocumentPicker en toont meteen hoeveel van de 20 MB uploadlimiet al gebruikt is. De backend controleert dezelfde limiet opnieuw, omdat frontendvalidatie alleen niet betrouwbaar genoeg is.

Bestanden worden lokaal op de API-server opgeslagen. De databank bewaart enkel metadata zoals bestandsnaam, content type, relatief bestandspad, ticket en bericht. Afbeeldingen krijgen een preview in de conversatie. Daardoor kan een gebruiker screenshots bekijken zonder ze eerst te downloaden.



Figuur 8.1: Attachment picker met geselecteerde bestanden, uploadlimiet en verwijderknoppen.

8.3. Documentatie

De documentatie is onderdeel van het eindresultaat. De gewone README beschrijft hoe het project gestart wordt. De technische README legt architectuur, workflows en belangrijke keuzes uit. Op enkele plaatsen in de code staan comments waar de intentie niet vanzelfsprekend is.

Deze documentatie ondersteunt onderhoudbaarheid. Een volgende ontwikkelaar moet niet alleen zien wat de code doet, maar ook waarom bepaalde keuzes gemaakt zijn.

9

Resultaten en evaluatie

Dit hoofdstuk evalueert het eindresultaat van Forcebit Ticketing. De vorige hoofdstukken beschreven vooral hoe de applicatie is opgebouwd. Hier ligt de nadruk op wat de applicatie oplevert, welke keuzes bijdragen aan onderhoudbaarheid en welke verbeteringen logisch zijn voor een volgende versie.

9.1. Werkend resultaat

De applicatie ondersteunt de basisflow waarvoor ze gebouwd werd. Een klant kan zich registreren, inloggen, een ticket aanmaken, antwoorden plaatsen, bijlagen toevoegen, een ticket sluiten en een gesloten ticket opnieuw openen. Een administrator kan tickets opvolgen, beantwoorden, zoeken, filteren en van status veranderen. Daardoor staat de communicatie niet meer verspreid over losse mails of telefoongesprekken: vraag, antwoord, status en bijlagen blijven samen in hetzelfde ticket. De statusflow is duidelijker geworden door de keuze voor **New**, **Open** en **Closed**. **New** betekent dat support nog niet gereageerd heeft. **Open** betekent dat de conversatie loopt. Een heropend ticket wordt opnieuw **Open**, omdat het al een geschiedenis heeft. Deze regel maakt het overzicht logischer dan wanneer elk heropend ticket opnieuw als nieuw zou verschijnen.

Mentorfeedback heeft vooral de bruikbaarheid verbeterd. De conversatie kreeg een chatweergave, afbeeldingsbijlagen kregen een preview en overzichten kregen zoekvelden, filters en paginatie. Zo blijft de basisworkflow eenvoudig, maar werkt de applicatie beter met realistische testdata.

9.2. Onderhoudbaarheid

Een belangrijk resultaat is dat de applicatie niet alleen werkt, maar ook een duidelijke structuur heeft. In de backend hebben controllers, services, repositories, domain rules en DTOs elk een eigen verantwoordelijkheid. Daardoor is het meestal

duidelijk waar een wijziging thuishoort. Een controller behandelt HTTP, een service voert de workflow uit, een repository haalt data op en een domain rule bewaakt een businessregel.

Die scheiding maakte wijzigingen tijdens het project eenvoudiger. De statusregels konden aangepast worden in de domain rules en service-laag. Bijlagen werden toegevoegd zonder de ticketlogica afhankelijk te maken van concrete bestandsopslag, omdat opslag achter `IFileStorageService` zit. Ook e-mail blijft apart: `TicketService` beslist wanneer een notificatie nodig is, terwijl `EmailService` de providergerichte details afhandelt.

De frontend volgt hetzelfde idee. Screens tonen pagina's, hooks beheren gedrag, API modules praten met de backend en componenten bevatten herbruikbare UI. De generieke paginatiehook toont het voordeel daarvan: dezelfde logica wordt gebruikt voor gesprekken, tickets en klantenlijsten. Documentatie en gerichte comments ondersteunen dit, omdat ze uitleggen waarom bepaalde keuzes gemaakt zijn en niet alleen wat de code doet.

9.3. Beperkingen en verdere ontwikkeling

De huidige versie blijft een prototype en geen volledige productieomgeving. Bijlagen worden lokaal op de API-server opgeslagen. Dat is voldoende voor ontwikkeling en demo, maar voor productie zou een aparte bestandsserver of object storage beter zijn. Dan kunnen back-ups, toegangsrechten, schaalbaarheid en opslaglimieten apart beheerd worden. De bestaande interface `IFileStorageService` maakt zo'n vervanging technisch haalbaar.

Ook het adminbeheer kan verder groeien. Op dit moment kan een administrator alle tickets beheren. In een grotere organisatie zou het nuttig zijn om administrators te koppelen aan categorieën, onderwerpen of concrete tickets. Dan kan een ticket automatisch bij de juiste persoon of het juiste team terechtkomen.

Voor grotere datasets zou server-side paginatie beter zijn dan alleen client-side paginatie. De frontend haalt dan enkel de nodige pagina op in plaats van de volledige lijst. Daarnaast zouden automatische tests rond statusregels, autorisatie, uploads en notificaties extra zekerheid geven bij toekomstige wijzigingen.

Omdat de frontend in Expo React Native geschreven is, blijft verdere ontwikkeling richting een mobiele applicatie mogelijk. De huidige webversie gebruikt al React Native componenten en schermlogica. Voor een mobiele versie zouden vooral navigatie, layoutdetails en platformafhankelijke interacties verder getest moeten worden. Samen met betere logging, sterker secret management en eventueel een audit trail zijn dat logische stappen richting langdurig gebruik.

10

Conclusie

De centrale onderzoeksvraag was hoe een onderhoudbare ticketingapplicatie gebouwd kan worden waarin klant en administrator een duidelijke supportworkflow volgen, met aandacht voor Single Responsibility, gelaagde structuur en herbruikbare frontendlogica. Het antwoord is dat onderhoudbaarheid vooral ontstaat door verantwoordelijkheden consequent te scheiden. In de backend krijgen HTTP, use cases, databanktoegang en businessregels elk een vaste plaats: controllers, services, repositories en domain rules. DTO's zorgen ervoor dat data gecontroleerd tussen lagen en API wordt doorgegeven.

Ook de deelvragen sluiten daarbij aan. De statusflow met **New**, **Open** en **Closed** wordt in de backend bewaakt, zodat een nieuw, lopend, heropend of afgesloten ticket overal dezelfde betekenis heeft. De frontend blijft leesbaar doordat screens, hooks, API modules en herbruikbare componenten apart blijven. Daardoor konden zoeken, filtering, paginatie, uploads en bijlagen toegevoegd worden zonder dat alle logica in de schermen terechtkwam.

De documentatie ondersteunt die structuur door de belangrijkste keuzes, verantwoordelijkheden en uitbreidingspunten vast te leggen. Mijn conclusie is daarom dat Forcebit Ticketing het communicatieprobleem oplost en aantoont dat een supportapplicatie onderhoudbaar kan blijven wanneer functionaliteit, regels en verantwoordelijkheden bewust gescheiden worden.

Kritisch bekeken blijft het project een proof of concept. Voor echt dagelijks gebruik zijn onder andere automatische tests, productiegerichte opslag en verdere uitwerking van beheerrollen nodig. De applicatie toont wel aan dat de gekozen workflow werkt en dat de technische basis duidelijk genoeg is om later verder op te bouwen.

Bibliografie

- Atlassian. (2026). *IT Ticketing Systems: Simplify Issue Tracking & Resolution*. Verkregen 5 juni 2026, van <https://www.atlassian.com/itsm/service-request-management/it-ticketing-system>
- AXELOS. (2019). *ITIL Foundation: ITIL 4 Edition*. The Stationery Office.
- Brevo. (2026). *Brevo API Documentation*. Verkregen 5 juni 2026, van <https://developers.brevo.com/docs/getting-started>
- Expo. (2026). *Expo documentation*. Verkregen 5 juni 2026, van <https://docs.expo.dev/>
- Formik. (2026). *Formik documentation*. Verkregen 5 juni 2026, van <https://formik.org/>
- Fowler, M. (2003). *Data Transfer Object*. Verkregen 5 juni 2026, van <https://martinfowler.com/eaCatalog/dataTransferObject.html>
- ISO/IEC. (2018). *ISO/IEC 20000-1:2018: Service management system requirements* (International Standard ISO/IEC 20000-1:2018). International Organization for Standardization. Verkregen 5 juni 2026, van <https://www.iso.org/standard/70636.html>
- Jacobson, I., Christerson, M., Jonsson, P. & Overgaard, G. (1992). *Object-Oriented Software Engineering: A Use Case Driven Approach*. ACM Press; Addison-Wesley.
- Jones, M. B., Bradley, J. & Sakimura, N. (2015). *JSON Web Token (JWT)*. Verkregen 4 juni 2026, van <https://www.rfc-editor.org/rfc/rfc7519>
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Meta Open Source. (2026). *React Native documentation*. Verkregen 4 juni 2026, van <https://reactnative.dev/docs/getting-started>
- Microsoft. (2026a). *ASP.NET Core documentation*. Verkregen 4 juni 2026, van <https://learn.microsoft.com/aspnet/core/>
- Microsoft. (2026b). *C# Enumerations*. Verkregen 5 juni 2026, van <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/enums>
- Microsoft. (2026c). *Entity Framework Core documentation*. Verkregen 4 juni 2026, van <https://learn.microsoft.com/ef/core/>
- Microsoft. (2026d). *Options pattern in .NET*. Verkregen 5 juni 2026, van <https://learn.microsoft.com/en-us/dotnet/core/extensions/options>

OWASP Foundation. (2026). *Authorization Cheat Sheet*. Verkregen 4 juni 2026, van https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

Quense, J. (2026). *Yup documentation*. Verkregen 5 juni 2026, van <https://github.com/jquense/yup>

React Navigation. (2026). *React Navigation documentation*. Verkregen 4 juni 2026, van <https://reactnavigation.org/docs/getting-started>